

RESUMO DO MÓDULO

Complemento - JOIN

Aqui vamos analisar os mais variados tipos de JOIN que existem no banco de dados, com base em exemplos, e veremos os seus resultados. Dessa forma, poderemos comparar como cada um funciona e entender quando devemos usar cada tipo.

Preparando o ambiente

Para mostrar como os tipos de JOIN funcionam, precisamos criar duas tabelas que terão um relacionamento simples para que possamos "cruzar" os dados. O objetivo não é focar em boas práticas para definição de tabelas, então vamos criar somente duas tabelas com uma coluna chamada Nome, que será o vínculo entre elas.

```
CREATE TABLE carros_a (nome VARCHAR(50));
```

```
CREATE TABLE carros_b (nome VARCHAR(50));
```

Agora vamos adicionar alguns dados para começarmos a testar os tipos de JOIN.

```
INSERT INTO carros_a VALUES ('Corsa'), ('Opala'), ('Fusca'), ('Ferrari');
```

```
INSERT INTO carros_b VALUES ('Celta'), ('Onix'), ('Fusca'), ('Ferrari');
```

Tipos de JOIN

Inner Join

Esse é o tipo de JOIN mais conhecido e, como o diagrama ilustraria, exibe os registros que são comuns entre as duas tabelas.

```
SELECT ca.nome nome_a, cb.nome nome_b  
FROM carros_a ca  
INNER JOIN carros_b cb ON ca.nome = cb.nome;
```

Neste exemplo, são exibidos apenas os nomes que existem nas duas tabelas, carros_a e carros_b, com base na correspondência exata de valores entre as colunas nome de ambas.

Left Join

O LEFT JOIN exibe todos os registros que estão na tabela A (carros_a), mesmo que não existam os mesmos registros na tabela B (carros_b), além dos registros da tabela B que têm em comum com a tabela A.

```
SELECT ca.nome nome_a, cb.nome nome_b  
FROM carros_a ca  
LEFT JOIN carros_b cb ON ca.nome = cb.nome;
```

Com este comando, é possível ver todos os nomes da tabela carros_a, mesmo que alguns nomes da tabela carros_b não correspondam. Se um nome de carros_a não existir em carros_b, a consulta retornará NULL para o campo correspondente.

Right Join

Usando o RIGHT JOIN, a consulta exibe todos os registros que estão na tabela B (carros_b), mesmo que não existam os mesmos registros na tabela A (carros_a), além dos registros que a tabela A tem em comum com a tabela B.

```
SELECT ca.nome nome_a, cb.nome nome_b  
FROM carros_a ca  
RIGHT JOIN carros_b cb ON ca.nome = cb.nome;
```

Esse tipo de JOIN é ideal quando o interesse é visualizar todos os registros da tabela carros_b e

complementar com os dados da tabela carros_a quando existirem.

Full Outer Join

Uma consulta usando o FULL OUTER JOIN exibe todos os registros que existem na tabela A (carros_a) e na tabela B (carros_b), independentemente de haver correspondência entre elas.

```
SELECT ca.nome nome_a, cb.nome nome_b  
FROM carros_a ca  
FULL OUTER JOIN carros_b cb ON ca.nome = cb.nome;
```

Com o FULL OUTER JOIN, são exibidos todos os registros de ambas as tabelas. Se um nome existir apenas em carros_a, o campo correspondente de carros_b será NULL. O inverso também ocorre quando um nome está presente apenas em carros_b.

Considerações Finais

Conhecer como funciona cada tipo de JOIN ajuda a resolver muitos problemas de forma simples e eficiente, sem precisar reinventar a roda. Esses recursos são essenciais para trabalhar com dados que estão distribuídos em diferentes tabelas, permitindo relacionar e analisar informações de maneira integrada.
